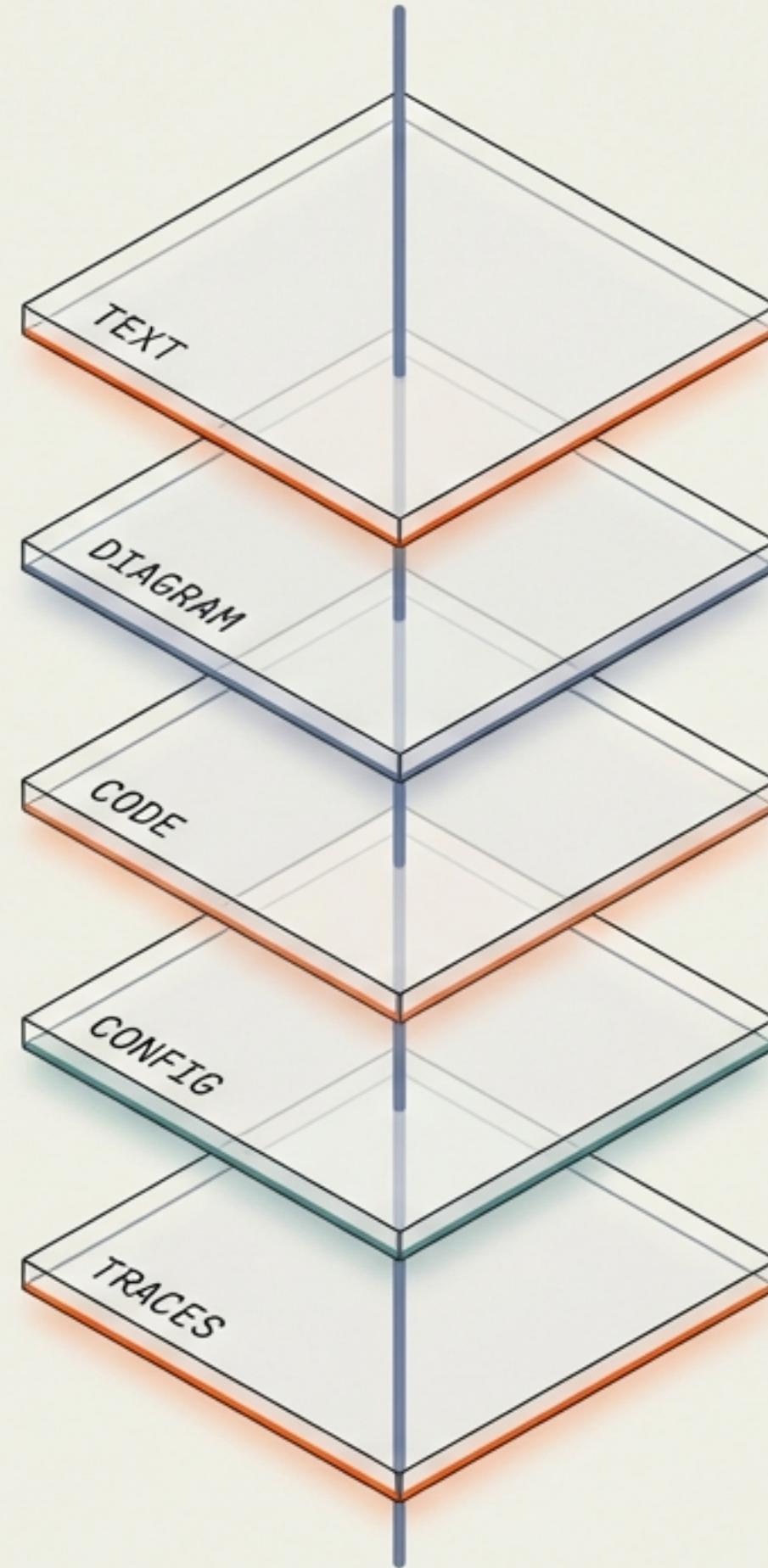


Compatibility Through Connectivity

A Paradigm Shift in System
Verification & Testing

BASED ON: ISSUES-FS v1.0
STATUS: FOUNDATIONAL DOCUMENT
DEPENDENCY: THINKING IN GRAPHS v1.0
DATE: 05 FEBRUARY 2026



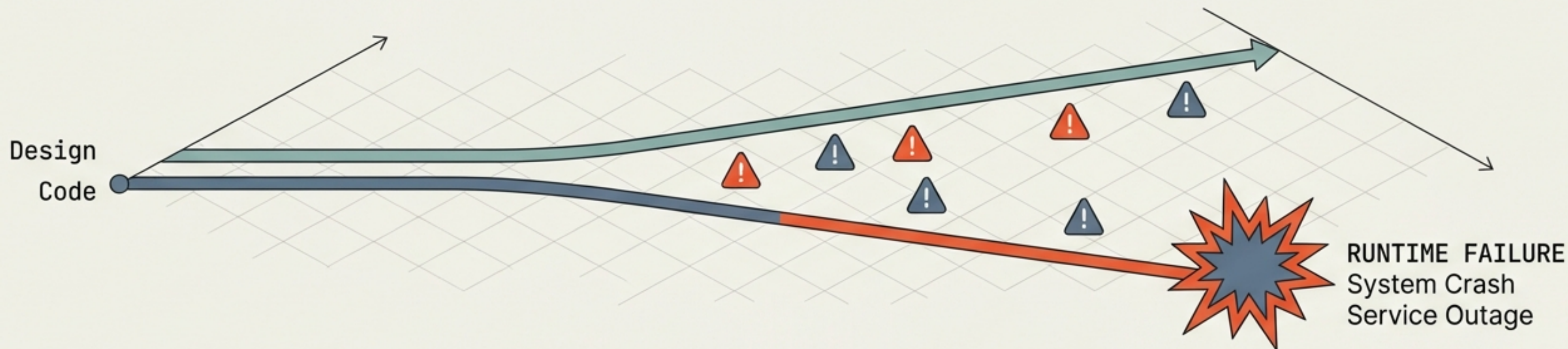
A system is not just code; it is a collection of fragmented descriptions.

A typical system is described by multiple authors using different languages. We treat these as separate artifacts, but they all claim to represent the same truth.

ARTIFACT	LANGUAGE	AUTHOR	PURPOSE
Architecture Docs	Natural Language (English)	Architect	Intent & Rules
Diagrams	Visual Language (Mermaid/Visio)	Architect	Structure & Flow
Code	Programming Language (Python/Go)	Developer	Implementation
Configs	Declarative (YAML/HCL)	DevOps	Deployment State
Traces	Behavioral Data (OpenTelemetry)	The System	Reality

We assume these representations align, but without verification, this is merely the “Illusion of Alignment.”

When representations diverge, drift accumulates silently until failure occurs.



Security Vulnerabilities



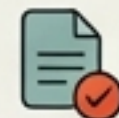
Phantom firewalls. The architect documents a boundary that is missing from the code.

Operational Surprises



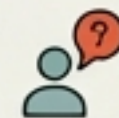
Runbooks describe failover processes that the active Kubernetes configuration does not support.

Audit Failures



Compliance claims "Data is encrypted at rest", but the Terraform implementation differs.

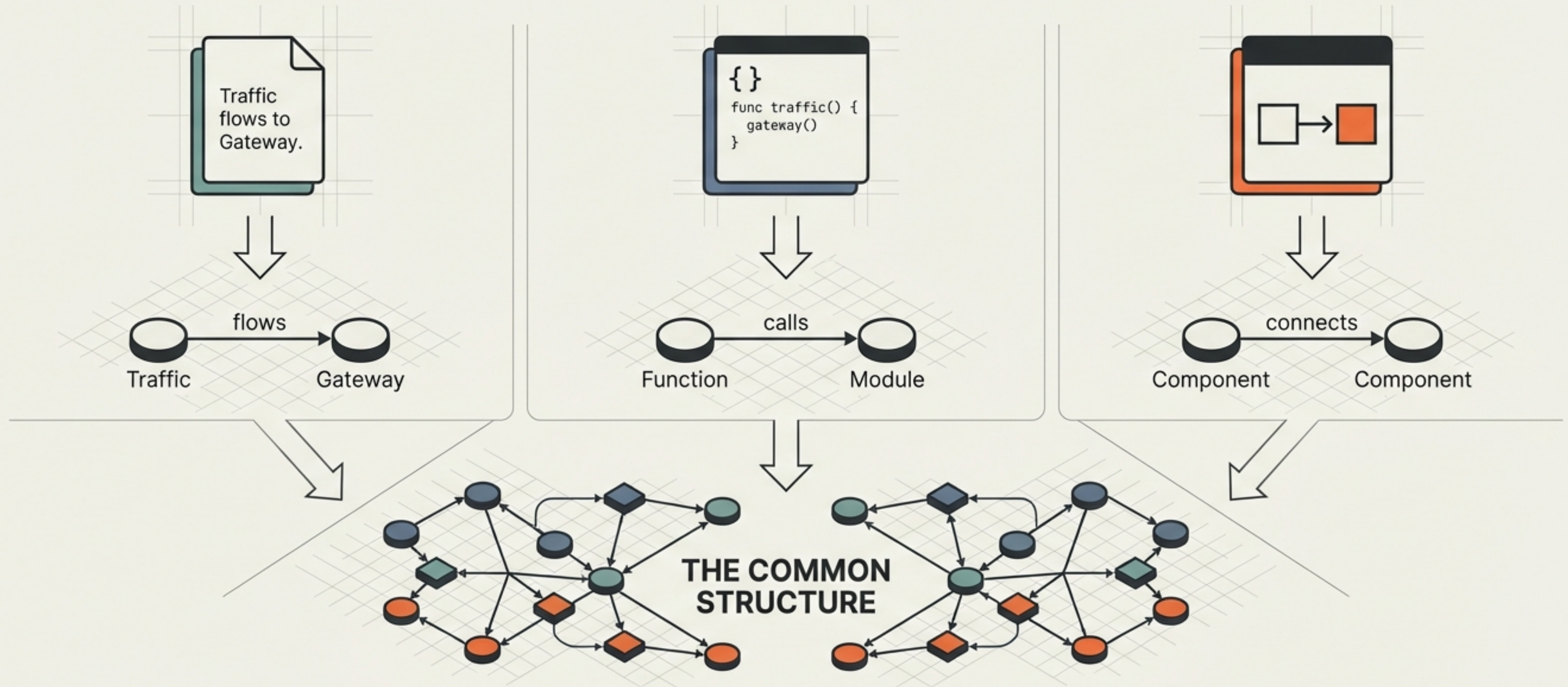
Confusion



New developers read the docs, look at the code, and find they describe two different systems.

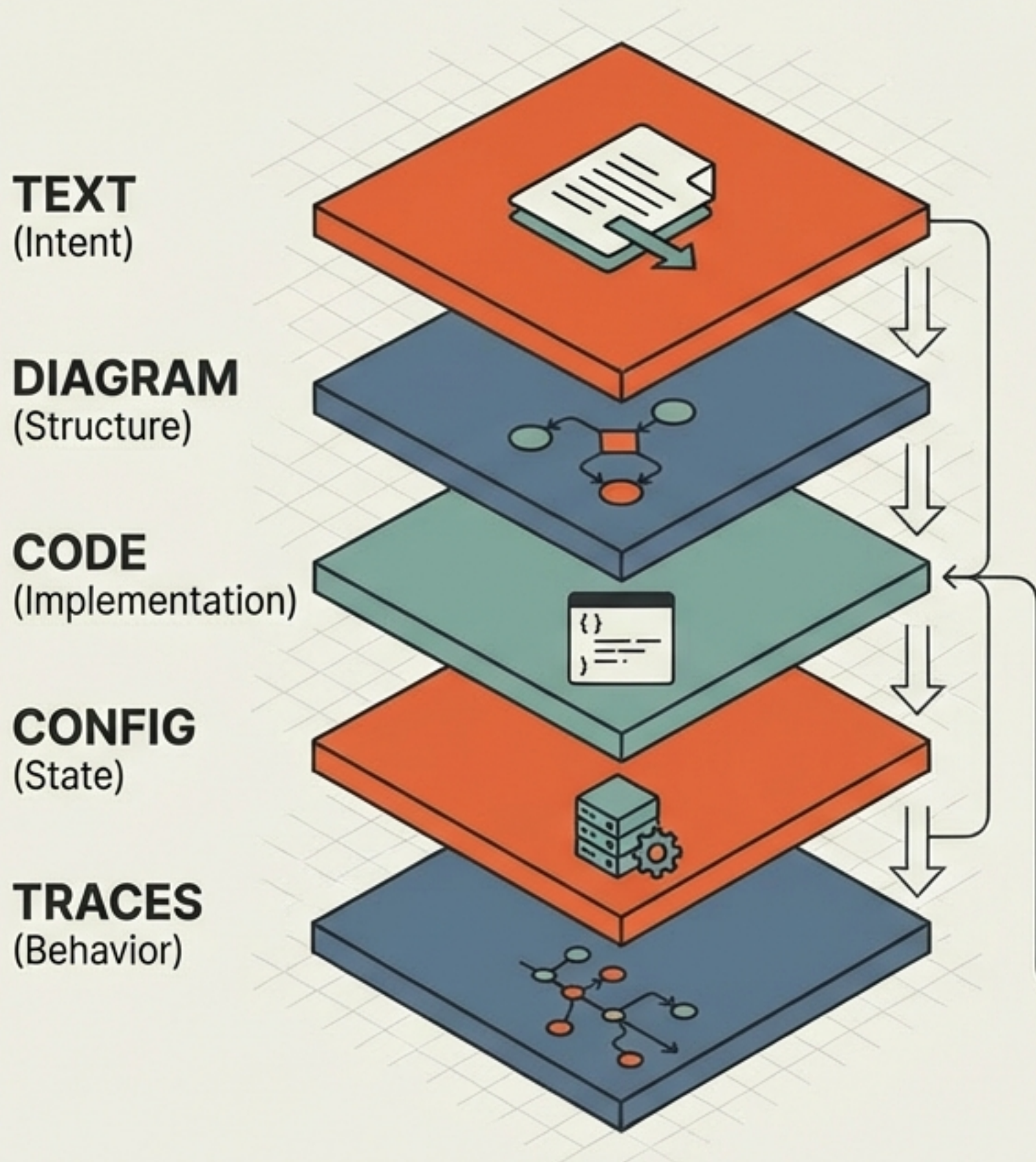
Misalignment is a primary source of system failure and security breaches.

The Unifying Theory: Every artifact is a graph.



Whether it is prose, ASCII art, Python, or YAML, every artifact describes nodes and relationships. This shared structure allows for mathematical comparison.

The Five Layers of System Reality



1. TEXT (Intent)

- **Source:** Architecture Docs, ADRs
- **Extraction:** NLP & Semantic extraction of rules ("must", "never").



2. DIAGRAM (Structure)

- **Source:** Mermaid, PlantUML, Images
- **Extraction:** Visual analysis of nodes, edges, and boundaries.



3. CODE (Implementation)

- **Source:** Python, Go, TypeScript
- **Extraction:** AST parsing, static analysis of call graphs.



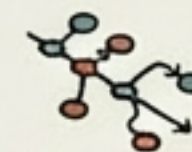
4. CONFIG (State)

- **Source:** Kubernetes, Terraform
- **Extraction:** Merging declared state vs. live API queries.



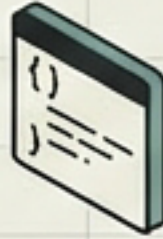
5. TRACES (Behavior)

- **Source:** OpenTelemetry, Logs
- **Extraction:** Analysis of request paths and spans.



Compatibility is not about correctness; it is about coherence.

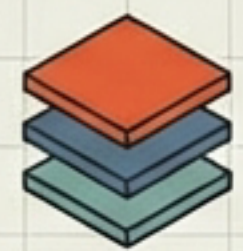
TRADITIONAL TESTING



Question: Does the code work?

Scope: Verifies Code against
Developer assumptions.

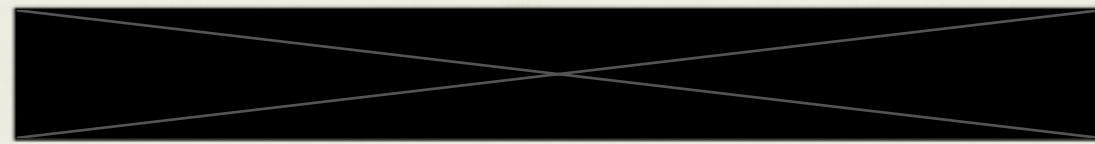
COMPATIBILITY TESTING



Question: Do all representations
agree?

Scope: Verifies Agreement across all
5 Layers.

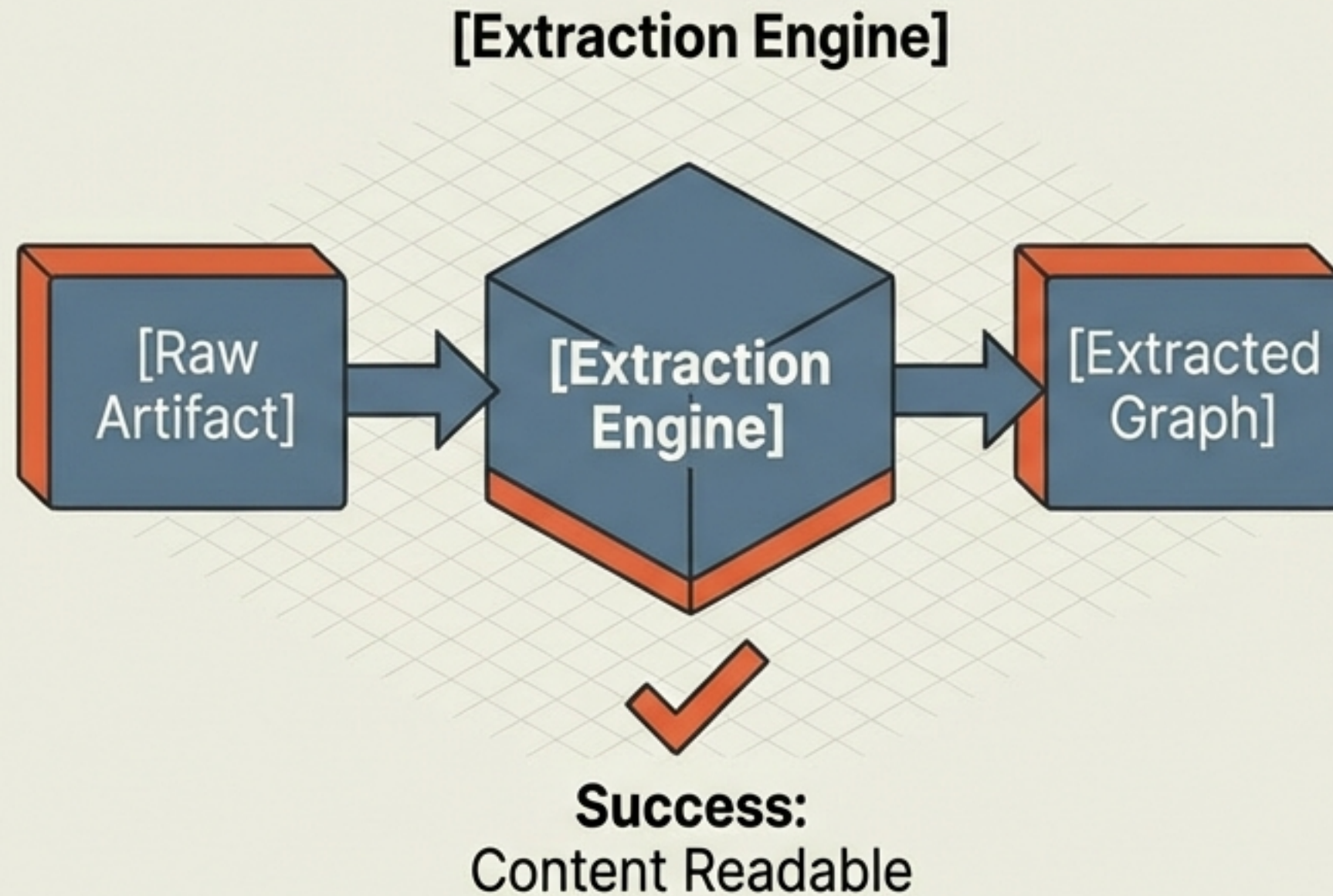
Compatibility = Connectivity Across Graphs



A Coherent System where Design = Implementation = Runtime.

Mechanism Part I: Extraction Tests

The “Always Pass” Rule



Extraction tests verify that we can accurately read an artifact. They do not judge whether the content is correct, only that we have understood it.

The **Rule**: Extraction tests always pass as long as the extraction logic functions.

PASS: 'We successfully extracted a rule that says the database has no password.'
(The system is insecure, but the extraction succeeded. The test passes.)

FAIL: "We could not parse the file format."

Mechanism Part II: Compatibility Tests

The Comparison Logic

1. INPUT:

Graph A (Text) + Graph B (Code)

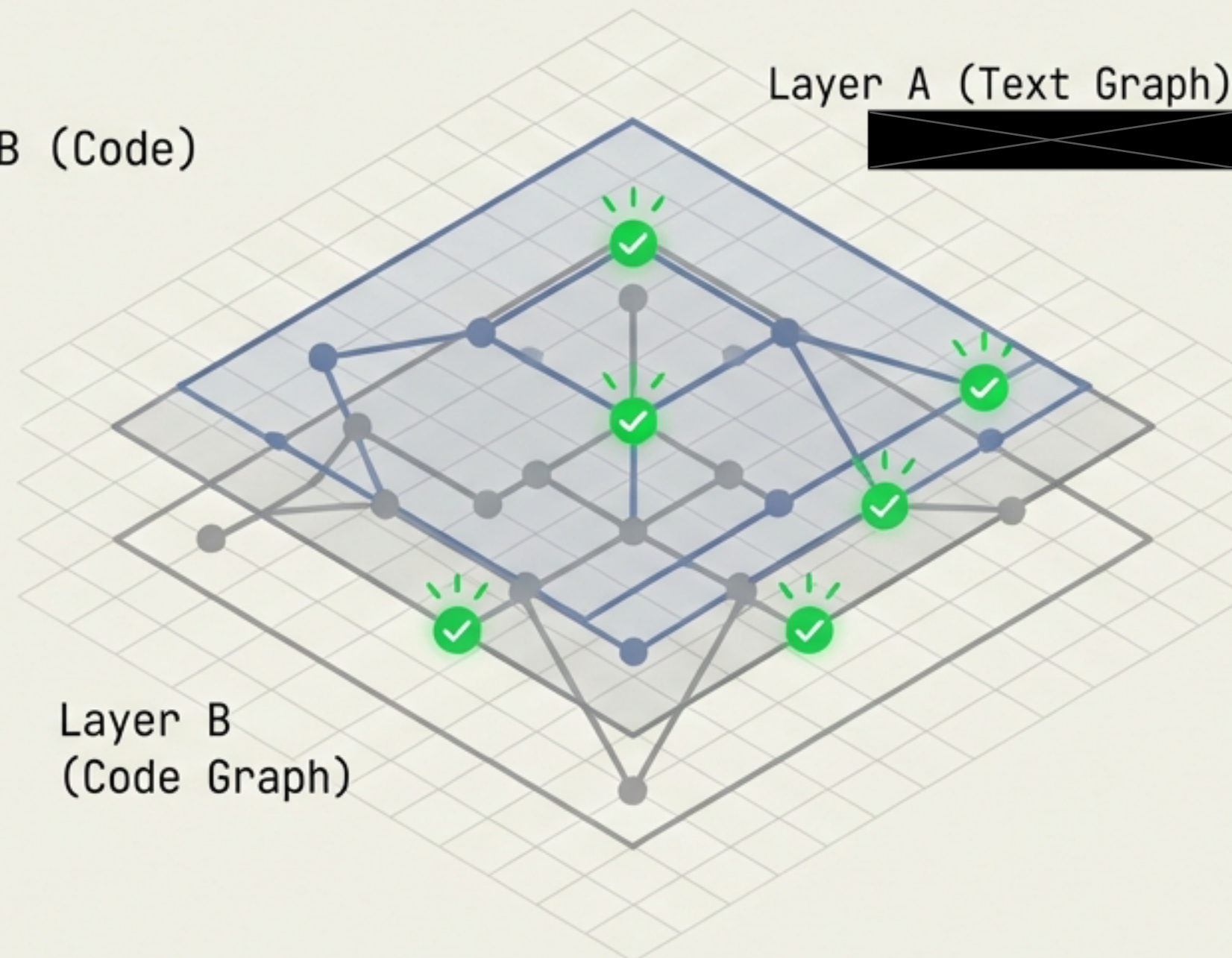
2. PROCESS:

Overlay nodes and edges based on the Lexicon (shared vocabulary).



3. CHECK:

Do the edges in Graph A exist in Graph B?



AGREEMENT CRITERIA CHECKLIST

- ✓ Both layers express the rule.
- ✓ The expressions are logically compatible.
- ✓ No contradictions exist.

Case Study: The Golden Thread

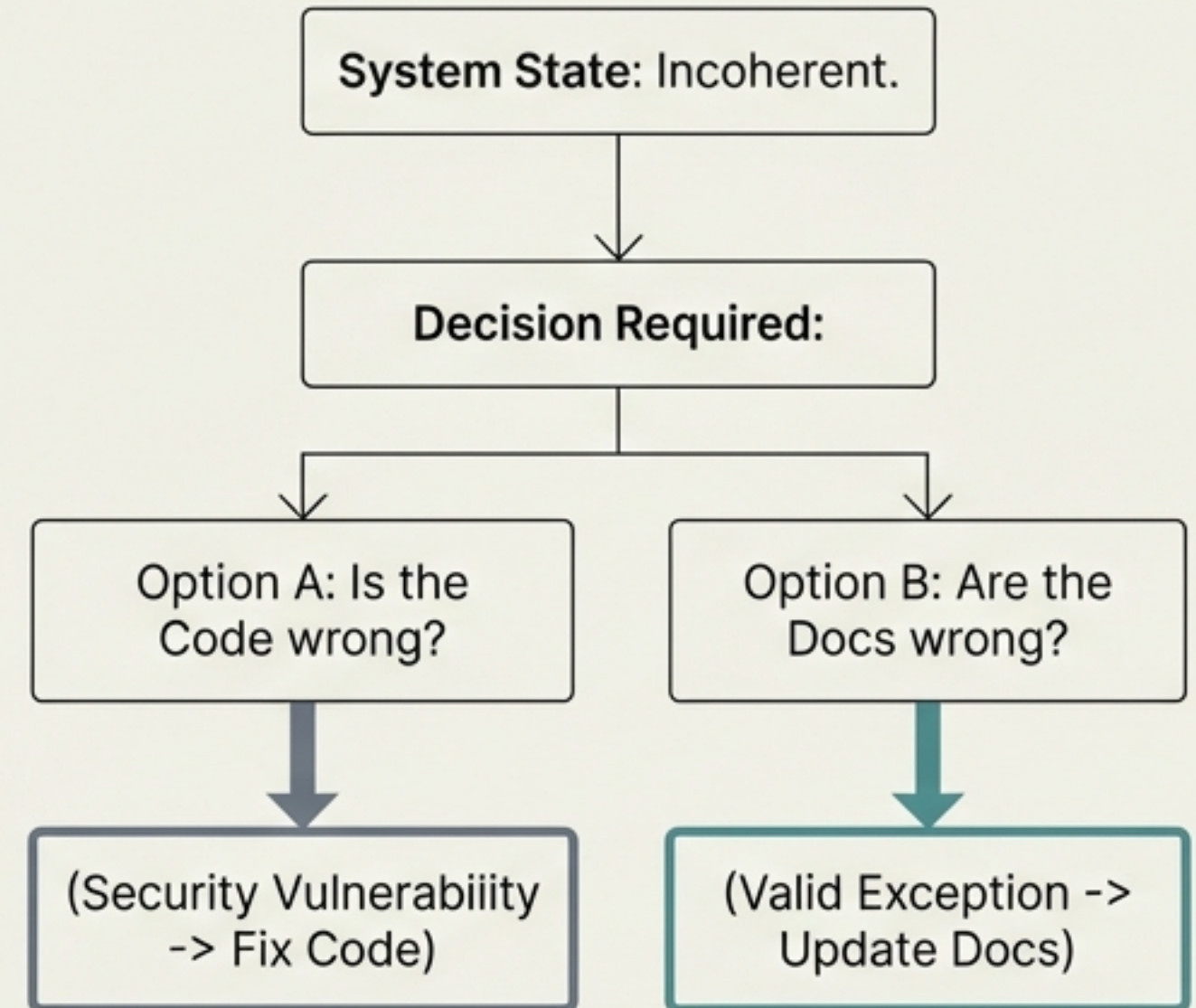
Rule: “All external traffic must pass through the API Gateway.”

THE VERIFICATION TABLE (HIGH DENSITY)			
LAYER	EXTRACTED DATA	STATUS	
Layer 1 (Text)	Subject(External) → Action(pass_through) → Object(Gateway) [REDACTED]	✓ EXPLICIT RULE	[VERIFICATION THREAD]
Layer 2 (Diagram)	Path: External → Gateway → Services (no bypass) [REDACTED]	✓ STRUCTURE ENFORCES	
Layer 3 (Code)	CallGraph: `external_handlers` calls `gateway_module` [REDACTED]	✓ IMPLEMENTATION MATCHES	
Layer 4 (Config)	State: Service `gateway` is type `LoadBalancer`. Others `ClusterIP`. [REDACTED]	✓ DEPLOYMENT ENFORCES	
Layer 5 (Traces)	Data: 100% of external requests contain `gateway_span` [REDACTED]	✓ RUNTIME CONFIRMS	

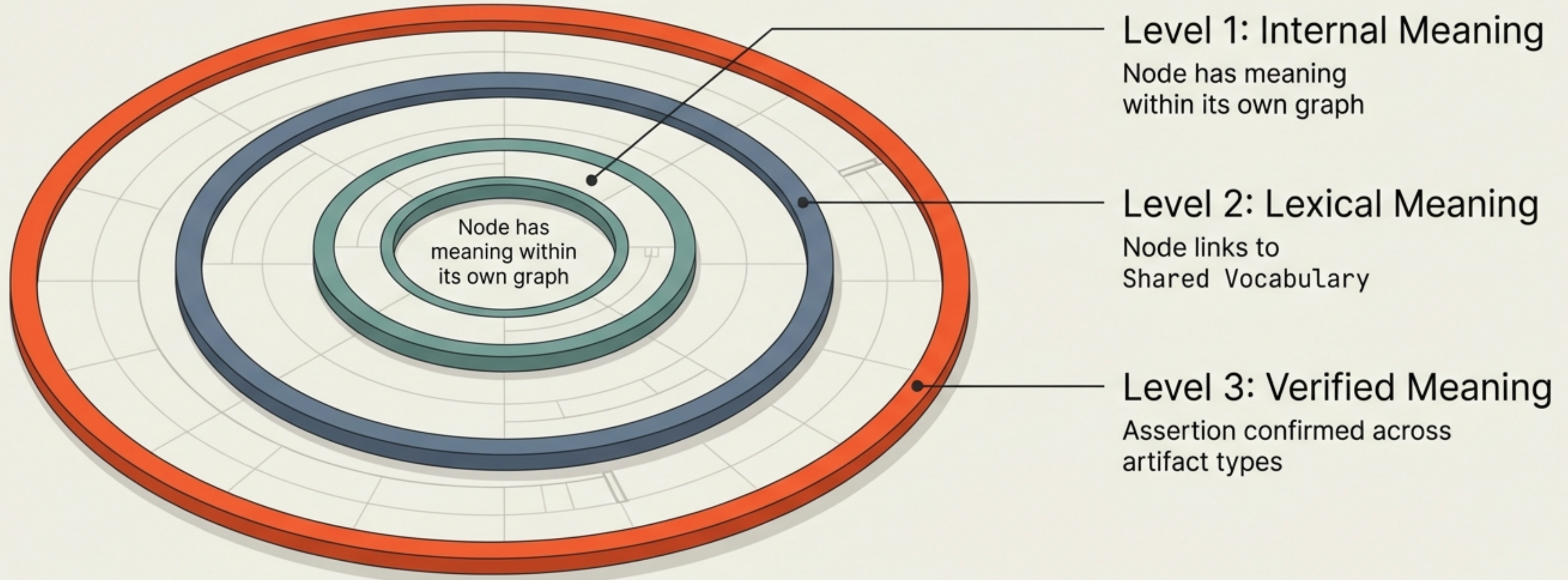
Handling Divergence: When the layers disagree.

Divergence is not failure; it is information.

Text/Diagram	Must pass through Gateway ✅ [AGREE]
Code	Admin_handler calls user_service directly ❗ [DIVERGENCE DETECTED]
Config	User-service exposes NodePort 30080 ❗ [DIVERGENCE DETECTED]
Traces	0.8% of traffic bypasses Gateway ❗ [DIVERGENCE DETECTED]



Meaning emerges from Cross-Graph Connectivity



A rule in a document is fiction until code and config confirm it.

Value Proposition by Role



Architect

My docs are no longer static text; they are executable specifications.



Developer

I can verify my implementation matches the architect's intent before I push.



DevOps

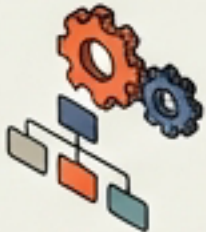
I can ensure that what is deployed matches what was designed and implemented.



Compliance

I can prove controls exist across all layers, replacing manual audit with automated verification.

From 'Working Code' to 'Coherent Systems'

		Traditional Workflow	Compatibility Workflow
1	Source of Truth	Diffused mental models 	The consensus of the graphs 
2	Test Derivation	Manual translation of requirements 	Automated derivation from Architecture Docs 
3	Drift Detection	Invisible; found during incidents 	Continuous; found in CI/CD 
4	Scope	Checks Code only 	Checks Text, Diagram, Code, Config, and Traces 

The 10 Principles of Compatibility

1. Every artifact is a graph.



2. Extraction tests confirm understanding.



3. Extraction tests always pass.



4. Compatibility tests compare graphs.



5. Divergence is information, not failure.



6. Meaning requires agreement.



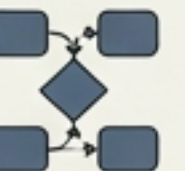
7. Layers have different vocabularies, but one structure.



8. Not all layers apply to all rules.



9. Compatibility is continuous (Run in CI/CD).



10. The goal is coherence.



Coherence is the metric of truth.

Currently, different people hold different mental models of the system. The architect's model lives in docs; the developer's in code; the ops model in runbooks.

Compatibility testing surfaces where these models diverge. It forces alignment. It turns the 'Illusion of Alignment' into verified reality.

